Project Report

Placement Portal Web Application

Author

Name: Suraj

Roll No: 24f3004030

Email: 24f3004030@ds.study.iitm.ac.in

Note: Currently pursuing Diploma Level, 2nd Term at IIT Madras (BS in Data Science).

How to Run

Follow these steps to set up and run the application:

1. Install the required dependencies: **flask** and **flask_sqlalchemy**
2. Run the **app.py** file using the **python app.py** command in your terminal.
3. Default admin credentials are created automatically: username **admin**, password **admin123**.

Description

The objective of this project is to build a Placement Portal web application using Flask as the backend framework. The portal connects students, companies, and an admin on a single platform. Students can browse and apply for placement drives. Companies can register, post drives, and manage applications. The admin oversees all activity, approves companies and drives, and can blacklist users if required.

Technologies Used

4. **Flask:** Handles request routing, rendering HTML templates, and defining all application views.
5. **Flask-SQLAlchemy:** Used for defining database models, running queries, and committing changes to the database.
6. **Jinja2:** Templating engine that adds dynamic content and logic to HTML files.
7. **SQLite:** Lightweight relational database used for storing all application data.
8. **Werkzeug:** Used for password hashing and verification to ensure secure authentication.

Database Schema Design

All tables have an **id** field as the primary key. The schema consists of five models:

1. Admin

- **id** — Integer, primary key
- **username** — String, unique, required
- **email** — String, unique, required
- **password_hash** — String, required

2. Company

- **id** — Integer, primary key
- **name** — String, unique, required
- **email** — String, unique, required
- **hr_contact** — String, required
- **website** — String, optional
- **approval_status** — String (Pending / Approved / Rejected)
- **is_blacklisted** — Boolean, default False

3. Student

- **id** — Integer, primary key
- **name**, **email**, **age**, **gender**, **phone** — All required
- **degree**, **skills**, **curriculum_vitae** — Optional
- **is_blacklisted** — Boolean, default False

4. PlacementDrive

- **id** — Integer, primary key
- **company_id** — Foreign key referencing Company
- **job_title** — String, required
- **job_description**, **eligibility_criteria** — Optional
- **application_deadline** — Date, required
- **status** — String (Pending / Approved / Rejected / Closed)

5. Application

- **id** — Integer, primary key
- **student_id** — Foreign key referencing Student
- **drive_id** — Foreign key referencing PlacementDrive
- **application_date** — DateTime, auto-set on creation
- **status** — String (Applied / Shortlisted / Selected / Rejected)

Architecture

The project follows the standard Flask application structure:

- **app.py** — Contains all route definitions and view logic.
- **model.py** — Separated module containing all database model definitions.
- **templates/** — Holds all HTML files, organized into Admin, Company, and Student subfolders.
- **static/** — Contains CSS files for styling.
- **instance/** — Stores the SQLite database file.

Features

Admin

- Login with a pre-created admin account (no self-registration).
- View a dashboard showing total students, companies, drives, and applications.
- Approve or reject company registrations and placement drives.
- Blacklist companies or students to restrict their access.
- Search for companies or students by name or email.

Company

- Register with name, email, HR contact, and website. Account requires admin approval before login.
- Post new placement drives with job title, description, eligibility criteria, and application deadline.
- Each posted drive requires admin approval before it becomes visible to students.
- View all applications received, and update each application status to Shortlisted, Selected, or Rejected.
- Close or delete existing drives from the dashboard.

Student

- Register with personal details including name, email, age, gender, phone, degree, skills, and CV link.
- Login and browse all approved placement drives from approved companies.
- Apply for drives with a single click. Each student can apply for a drive only once.
- Search for drives by company name, job title, or eligibility criteria.
- Track application statuses (Applied, Shortlisted, Selected, Rejected) from the dashboard.
- Update profile details at any time from the profile page.

API Routes

The application exposes the following key routes:

Authentication

- **GET/POST /student_register** — Student registration
- **GET/POST /student_login** — Student login
- **GET/POST /company_register** — Company registration
- **GET/POST /company_login** — Company login
- **GET/POST /admin_login** — Admin login
- **GET/POST /logout** — Clears session and logs out

Admin Actions

- **GET /approve_company/<id>** — Approve a company
- **GET /reject_company/<id>** — Reject a company
- **GET /blacklist_company/<id>** — Blacklist a company
- **GET /approve_drive/<id>** — Approve a placement drive
- **GET /reject_drive/<id>** — Reject a placement drive
- **GET /blacklist_student/<id>** — Blacklist a student

Company Actions

- **GET/POST /create_drives** — Post a new placement drive
- **GET /close_drive/<id>** — Close an active drive
- **GET /delete_drive/<id>** — Delete a drive
- **GET /update_application_status/<id>/<status>** — Update an application status

Student Actions

- **POST /apply_job/** — Apply for a placement drive
- **GET/POST /student_dashboard/student_profile** — View and update student profile

Video Demonstration

A video walkthrough of the application is available at the link below:

Video Link:
https://drive.google.com/file/d/1tcsWrVs40PCmP2U8Xb-iZABqqRB71QTg/view?usp=drive_link

AI Usage Disclosure

Around 12% of AI assistance was used in this project, mainly for resolving errors and getting unstuck at a few points during development. Whenever something was not working as expected or I could not figure out why a particular piece of code was breaking, I used AI to understand what was going wrong. Other than that, the logic, structure, and overall implementation was thought out and written by me. The idea, the flow, and most of the coding decisions were made on my own.